Briefly describe the artifact. What is it? When was it created?

The artifact selected for the Software Engineering and Design category was originally developed in IT145: Found in App Development. This is a Java-based rescue animal management application designed to support the intake, management and reservation of rescue animals. The application uses object-oriented programming concepts including classes, inheritance, encapsulation and collections. The application contains a base RescueAnimal class with Dog and Monkey subclasses and uses Java ArrayList collections to store animal information. The original applications functionality is concentrated in the Driver class. The Driver class displays the user interface, collects and validates input, creates animal objects, searches through the collections, processes reservations and directly manages the application's data. This artifact was then transformed during the software engineering and design enhancement to reduce coupling and increase modularity and separation of concerns.

Justify the inclusion of the artifact in your ePortfolio. Why did you select this item? What specific components of the artifact showcase your skills and abilities in software development? How was the artifact improved?

I selected this artifact because the rescue animal management system is fundamentally about storing and retrieving data, yet the original version kept everything in an in-memory ArrayList, so all information was lost whenever the program closed. That made it a natural fit for the database enhancement, and it let me demonstrate database fundamentals alongside the ability to build a complete solution from the ground up with security in mind. I designed a relational MySQL schema that mirrors the class hierarchy, connected it to the application through JDBC, and implemented a JdbcAnimalRepository that plugs into the existing layered architecture, so the

service and UI layers needed only minimal changes. The artifact shows my ability to implement full CRUD functionality with PreparedStatements, map result sets to Java objects, use transactions so related inserts succeed or fail together, and handle errors gracefully. I also documented the setup in a quick start guide so others can reproduce the project on their own systems. Compared to the original, the application now persists data between sessions, uses appropriate data types, validates user input, and restricts database access through a least-privilege account.

Did you meet the course outcomes you planned to meet with this enhancement in Module One? Do you have any updates to your outcome-coverage plans?

Yes, I met the course outcomes I planned in Module One, and I have no updates to my outcome-coverage plans

“Demonstrate an ability to use well-founded and innovative techniques, skills, and tools in computing practices for the purpose of implementing computer solutions that deliver value and accomplish industry-specific goals.”

This enhancement demonstrates this outcome by integrating MySQL, SQL and JDBC into an existing Java application to solve a practical limitation of the original system. Rather than storing rescue animal information only in Java collections, the enhanced application uses a persistent relational database, so information persists between sessions. The implementation shows practical use of industry-relevant database technologies to improve the functionality, reliability, and maintainability of an existing solution. In a real-world scenario, it would give the organization a permanent storage solution in place of its old system.

“Develop a security mindset that anticipates adversarial exploits in software architecture and designs to expose potential vulnerabilities, mitigate design flaws, and ensure privacy and enhanced security of data and resources.”

This outcome is demonstrated through parameterized SQL and JDBC PreparedStatement objects for all user-provided information, which prevents user input from being incorporated directly into executable SQL and reduces exposure to SQL injection. Database constraints (primary and foreign keys, a unique name, NOT NULL and CHECK rules) and a relational design protect data integrity and reduce the possibility of invalid or inconsistent records. Database errors are handled with try-catch blocks and a custom exception, so users see a generic message instead of raw database details. Credentials are kept in a configuration file instead of the source code, and the application connects through a dedicated account with only the permissions it needs, following the principle of least privilege.

Reflect on the process of enhancing and modifying the artifact. What did you learn as you were creating it and improving it? What challenges did you face?

This was the longest enhancement, mostly because of its volume rather than its difficulty. In my previous database courses, the environment was already set up in a virtual platform like Codio, so building and connecting a local environment was new to me. It took some research to learn which tools I needed and how to set them up: MySQL Server, MySQL Workbench, and Connector/J, followed by the imports, the application user and the properties file. Once that stack was running and I was writing actual SQL statements and queries, the work felt much more familiar.

The rest of the challenge was converting the codebase to work with persistent storage. I had to add much more validation logic to the console UI than I wanted to, but it was necessary to prevent type mismatches such as strings going into numeric or date columns, since everything in the previous build was a String. There was also a lot of SQL exception handling to account for, and error handling was the name of the game, as the try-catch blocks show. The biggest challenge was simply the volume of work, which I could have reduced by starting earlier and spreading the environment setup and the many prepared statements over more time. My biggest learning moments were configuring my first local MySQL environment and learning how prepared statements prevent SQL injection.